

Управление встроенным светодиодом на плате *STM32F103* с помощью программы *PicOS-LED*

Дата: 2025.10.10

Автор: Шендрик Павел Дмитриевич

Оглавление

Введение	1
Схема подключения	3
Описание программы	4
Работа программы	6
Список литературы.....	8

Введение

Для включения встроенного светодиода на плате *STM32F103 BluePill*, также дополнительно необходимы: программатор *STLINK V2*, преобразователь *USB-TO-UART* и программа *PicOS-LED*, которая в дальнейшем будет использоваться для решения более сложных задач. Для взаимодействия с программой рекомендуется использовать *Terminal v1.9b*.



Рис. 1 *STM32F103 BluePill*, *ST-LINK V2* и *CH340* (*USB-TO-UART* преобразователь)

При разработке цифровых систем управления, как правило, используются микроконтроллеры (далее «МК»), которые выполняют целый ряд определённых задач. Большая часть недорогих МК имеют одно вычислительное «ядро», которое выполняет заданную программу последовательно, шаг за шагом. Однако, задачи обычно требуется выполнять одновременно — генерировать импульсы, обмениваться данными с устройствами, выполнять опрос датчиков, анализировать данные, проводить измерения с помощью АЦП и так далее.

Таким образом, при написании программ для МК приходится обеспечивать так называемую «псевдомногозадачность». Поэтому предлагается использовать автоматное программирование [1].

В основе автоматного программирования лежит концепция разбиения всех задач на малые подзадачи, каждая из которых не требует большого количества времени на своё выполнение.

При выполнении бесконечного цикла `while()` в главной функции `main()` происходит попеременный вызов функций (конечных автоматов), каждая из которых представляет собой пошаговую реализацию своей конкретной задачи. Внутри каждого такого автомата, в момент его вызова, происходит выполнение только одной подзадачи, необходимой для последовательного выполнения всей задачи.

Конечные автоматы оформлены в форме конструкции `switch()`, а подзадачи - в виде их `case`'ов. При этом, для соблюдения последовательного выполнения подзадач, вводится переменная с припиской `state`, которая и используется в конструкции `switch()`. Схематичное представление изображено на рисунке 2.

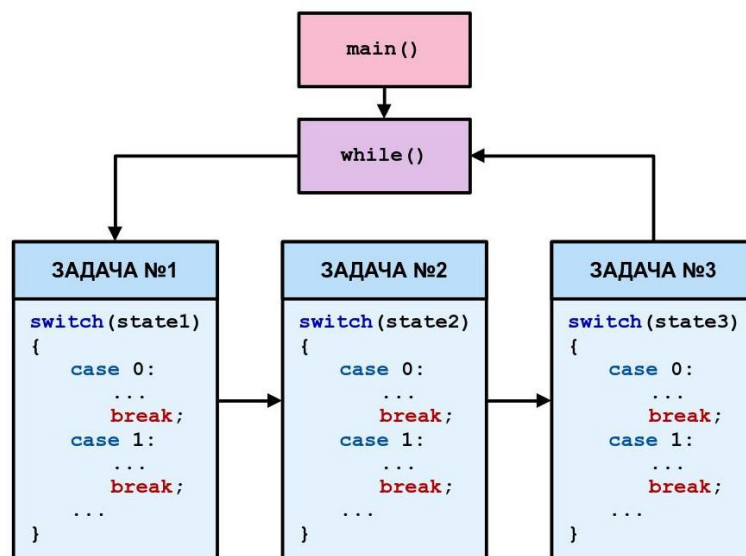


Рис. 2 Концепция автоматного программирования

Схема подключения

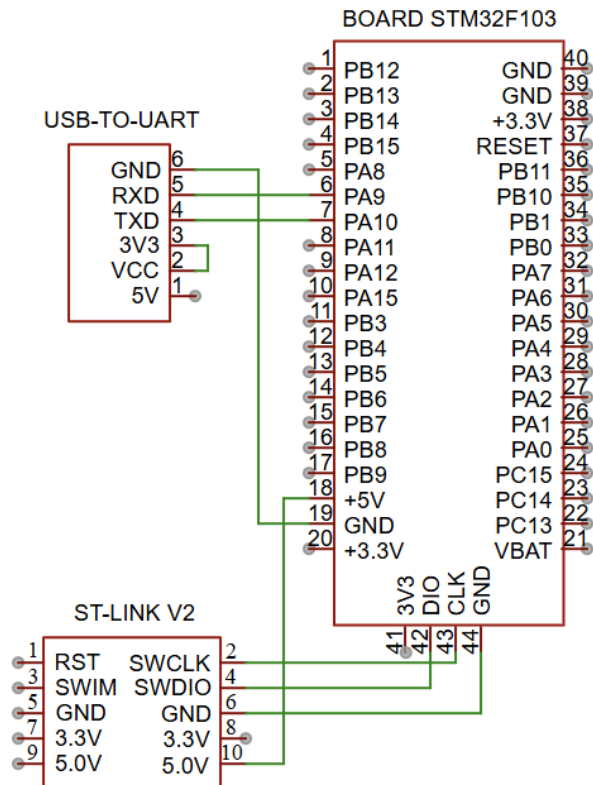


Рис. 3 Схема подключения

Названия выводов 41 – 44 на плате, могут отличаться от схемы, поэтому необходимо подключать контакты *ST-LINK V2* к соответствующим контактам платы *STM32F103*: *SWCLK* к *SWCLK* (или его сокращению *CLK* и т.п.); *SWDIO* к *SWDIO* (или его сокращению *DIO*, *D* и т.п.); *GND* к *GND* (или его сокращению *G* и т.п.); *5V* к *5V*.

Описание программы

PicOS – простейшая операционная система (далее «ОС»), обеспечивающая минимальный набор базовых функций. Программа состоит из конечных автоматов, постоянно взаимодействующих друг с другом. Главной задачей ОС является общение между системой и пользователем: обработка полученных строк, ответ на команды, вывод и изменение переменных.

В главном цикле *while* (рис. 4) находятся вставки *#include* с содержимым всех автоматов. Такой способ представления информации используется для экономии места, улучшения читаемости главного цикла и ускорения работы системы – во время компиляции программы, вставка заменяется на соответствующий файл, что приравнивается к копированию всего содержимого автомата в *while*, а использование функций, вместо вставок сказывается на быстродействии (т.к. используются вызовы и возвраты из функций), что очень критично для встраиваемых систем. Все автоматы находятся в папке «Tasks», они содержат собственное краткое описание и комментарии к коду.

```
// ===== ГЛАВНЫЙ ЦИКЛ =====  
while (1) {  
    #include "Tasks/usart1_send.c" // Отправка данных по USART1  
    #include "Tasks/usart1_recv.c" // Принятие строки по USART1  
    #include "Tasks/picos.c" // Реализация PicOS  
    #include "Tasks/var_get.c" // Вывод значений переменных по USART1  
    #include "Tasks/var_set.c" // Изменение значений переменных  
    #include "Tasks/led_switch.c" // Изменение состояния светодиода PC13  
}
```

Рис. 4 Главный цикл программы

Все автоматы можно условно разделить на системные, главные и пользовательские:

- Системные автоматы (*usart1_recv*, *usart1_send*, *var_get*, *var_set*) выполняют функции операционной системы: обработку ввода/вывода, управление переменными.
- Главный автомат (*picos*) опрашивает и управляет работой других автоматов, а также анализирует полученные команды.
- Пользовательский автомат (*led_switch*) реализует конкретную функцию управления по пользовательским командам.

Таким образом, шестой автомат служит для включения и выключения встроенного светодиода *PC13*. Включение светодиода произойдёт после получения команд «led 1» или «led = 1», а выключение – после получения «led 0» или «led = 0». Все команды изменяют или запрашивают значения переменных (*led*, *sensor* и т.п.), которые и отслеживаются в программе.

Все переменные, которые можно изменить или запросить командой, описываются в таблице переменных *var_table[]* (рис. 5), каждый элемент таблицы представляет собой структуру из четырёх элементов: Отображаемое имя переменной *name[12]*, краткое описание переменной *description[32]*, тип переменной *type* и указатель на саму переменную **ptr*. Таким образом таблица из переменных *version* и *led* будет выглядеть так:

```
// ===== Инициализация таблицы переменных =====
var_table_t var_table[] = {
    {"version", "Version of the firmware", TYPE_UINT16_T, &fm_version},
    {"led", "LED PC13 state: 1 - ON, 0 - OFF", TYPE_UINT16_T, &led_status}
};
```

Рис. 5 Таблица переменных *var_table[]*

Где, *version* – версия программы (равна 31, используется для преобразования в 3.1);

led – статус светодиода: ≥ 1 – включен, 0 – выключен;

Таблицу переменных *var_table[]* можно найти в файле «main.c» или вывести командой «help».

Работа программы

После запуска программы автоматы *usart1_send* и *usart1_recv* бездействуют, т.к. их состояния «*usart1_..._state*» равны «*USART1_..._NOT_ACTIVATED*». Первым запущившимся автоматом будет *picos*, который запустит *usart1_send*, для вывода приветственного сообщения, и *usart1_recv*, для получения команд пользователя. По умолчанию состояния автоматов *var_get* и *var_set* также равны «*..._NOT_ACTIVATED*», и они будут неактивны. Их работа регулируется автоматом *picos*, после получения соответствующих команд от пользователя.

Например, после получения автоматом *usart1_recv* команды «*led 1*» и её последующей обработки автоматом *picos*, он запустит автоматы *var_set*, который изменит значение переменной, и *var_get*, который и выведет изменённую переменную с её новым значением. Из-за изменения переменной *led*, сработает автомат *led_switch* и включит встроенный светодиод *PC13*.

На самом деле программа сложнее, т.к. обработка строки происходит за несколько циклов программы, а светодиод включится раньше, чем пользователь получит строку с новым значением переменной.

Для запуска программы:

1. Соберите схему (рис. 6);
2. Вставьте *CH340* и *ST-LINK V2* в компьютер и подключитесь к новому *COM*-порту в приложении *Terminal*;
3. Перепрограммируйте МК запустив *_comp+prog.bat*, чтобы в *Terminal* вывелось приветственное сообщение «Welcome to PicOS v3.1».

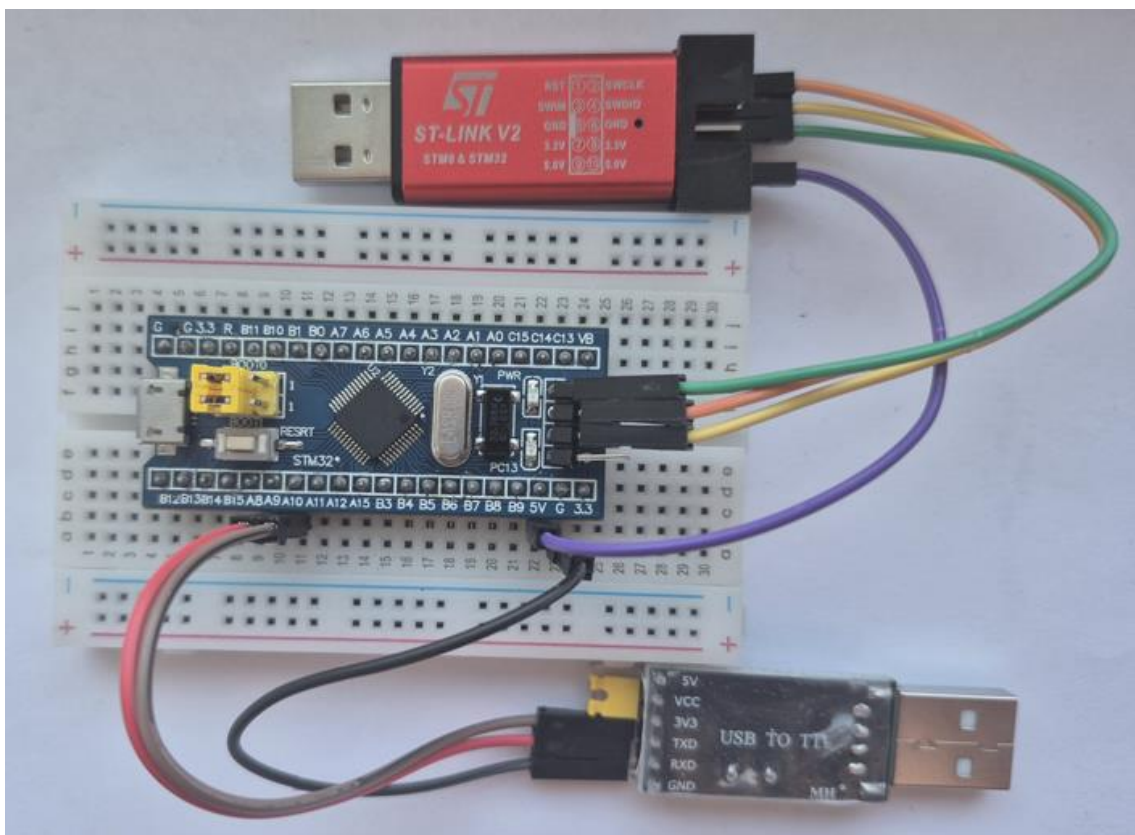


Рис. 6 Собранная схема

Для введения команд рекомендуется использовать окно посимвольного ввода в программе *Terminal* (самое нижнее). Все введенные команды требуют подтверждение – получение системой специального управляющего символа «+CR (Carriage Return)», который можно отправить нажатием на *Enter*, поэтому команду до подтверждения можно исправлять. Корректная работа программы и настройки *Terminal* изображены на рисунке 7.

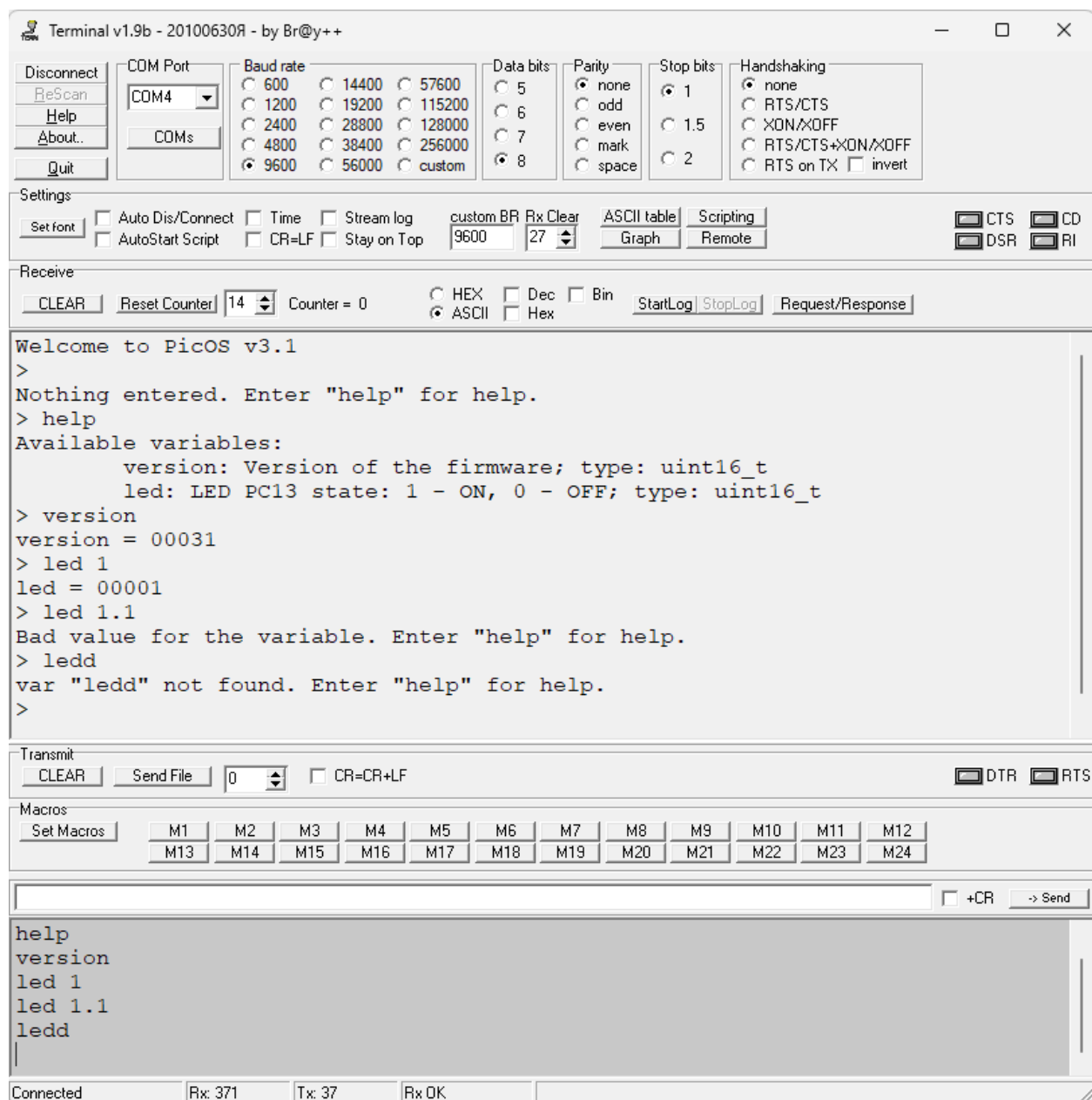


Рис. 7 Результаты работы программы *PicOS-LED* и настройки *Terminal*

Список литературы

1. Введение в автоматное программирование микроконтроллеров STM32 / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2024. –Т. 2. –No 4. –С. 21–25. DOI:10.18799/29495407/2024/4/75. URL: <https://indcyb.ru/journal/article/view/75/60> (дата обращения: 01.02.2025).
2. Пример псевдопараллельного выполнения трёх задач на микроконтроллере STM32F103 без использования операционной системы / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2025. –Т. 3. –No 2. –С. 1–7. DOI:10.18799/29495407/2025/2/88 // URL: <https://indcyb.ru/journal/article/view/88/71> (дата обращения: 30.06.2025).